

Video 2.1: What Privileged Containers Are and Why They're Dangerous

Duration: 8:00

Format: PPT (25%) / Terminal (40%) / Code Editor (20%) / Browser (15%)

Resolution: 1920x1080 @ 30fps

Prerequisites: Module 1 complete

SCRIPT BEGIN

[0:00-0:50] — What Does `--privileged` Actually Do? (PPT Slide)

SLIDE: Title: "The `--privileged` Flag — What It Unlocks"

Two-column comparison:

Default Container	Privileged Container
14 capabilities	ALL 41 capabilities
Seccomp filter active (~44 calls blocked)	Seccomp DISABLED
AppArmor profile applied	AppArmor DISABLED
Device access denied	ALL host devices accessible
/proc and /sys limited	/proc and /sys fully writable
Read-only cgroup filesystem	Writable cgroup filesystem
Cannot load kernel modules	CAN load kernel modules
Cannot mount filesystems	CAN mount ANY filesystem

NARRATION:

"The `--privileged` flag is a single switch that disables virtually every security mechanism Docker applies to a container. Let me be precise about what it does. It grants all 41 Linux capabilities — not just the default 14. It disables the Seccomp system call filter entirely. It disables AppArmor or SELinux confinement. It gives the container access to every device on the host — every disk, every USB device, every GPU. It makes `/proc` and `/sys` fully writable, which means the container can reconfigure kernel parameters. It makes the cgroup filesystem writable, allowing the container to modify its own resource limits. And it grants the ability to load kernel modules — injecting code directly into the kernel. In short, `--privileged` gives the container the same level of access as a root process running directly on the host. There is no meaningful isolation left."

[0:50-2:00] — Proving the Difference (Terminal)

SCREEN: Terminal at 150% zoom

NARRATION:

“Let me prove each of these differences.”

COMMAND + EXPLANATION:

```
# Compare capabilities: default container
$ docker run --rm alpine sh -c \
    'apk add -q libcap && capsh --print | grep "Bounding set"'
```

“Default container — 14 capabilities in the bounding set.”

```
# Compare capabilities: privileged container
$ docker run --rm --privileged alpine sh -c \
    'apk add -q libcap && capsh --print | grep "Bounding set"'
```

“Privileged container — every single capability is present. Let me highlight the dangerous additions.”

```
# Decode and compare
$ echo "=== DEFAULT ===" && \
  docker run --rm alpine sh -c \
    'cat /proc/1/status | grep CapBnd' && \
  echo "=== PRIVILEGED ===" && \
  docker run --rm --privileged alpine sh -c \
    'cat /proc/1/status | grep CapBnd'
```

“The capability bitmask for the privileged container has every bit set. That includes SYS_ADMIN — which alone can be used for container escape. It includes SYS_PTRACE — which allows attaching to any process. SYS_MODULE — for loading kernel modules. NET_ADMIN — for reconfiguring networking. DAC_READ_SEARCH — for bypassing file permission checks. Every one of these is an escalation vector.”

[2:00-3:00] — Seccomp and AppArmor Status (Terminal)

```
# Check Seccomp status: default container
$ docker run --rm alpine cat /proc/1/status | grep -i seccomp
Seccomp:      2
Seccomp_filters:  1
```

“Default container — Seccomp mode 2 with one filter loaded. System calls are being restricted.”

```
# Check Seccomp status: privileged container
$ docker run --rm --privileged alpine cat /proc/1/status | grep -i seccomp
Seccomp:      0
Seccomp_filters:  0
```

“Privileged container — Seccomp mode 0. Filtering is completely disabled. Every system call the kernel supports is available to this container. That’s over 350 system calls — including `mount`, `kexec_load`, `reboot`, `init_module`, and `bpf`.”

```
# Check AppArmor status: default container
$ docker run --rm alpine cat /proc/1/attr/current
docker-default (enforce)
```

“Default container — the docker-default AppArmor profile is enforced.”

```
# Check AppArmor status: privileged container
$ docker run --rm --privileged alpine cat /proc/1/attr/current
unconfined
```

“Privileged container — unconfined. No mandatory access control whatsoever.”

[3:00-4:15] — Device Access (Terminal)

NARRATION:

“Now let’s look at device access — this is where it gets really dangerous.”

```
# Default container – limited /dev
$ docker run --rm alpine ls /dev/ | wc -l
# Typically 15-20 devices
$ docker run --rm alpine ls /dev/
```

“A default container sees a minimal set of devices — null, zero, random, urandom, and a few pseudo-terminals. No disk devices, no USB, no GPU.”

```
# Privileged container – full host /dev
$ docker run --rm --privileged alpine ls /dev/ | wc -l
# Typically 100+ devices
$ docker run --rm --privileged alpine ls /dev/ | head -30
```

“A privileged container sees everything. Every device on the host is visible and accessible.”

```
# The critical one: host disk devices
$ docker run --rm --privileged alpine ls /dev/sd*
/dev/sda /dev/sda1 /dev/sda2
```

“There are the host’s disk devices — sda, sda1, sda2. This is the host’s primary hard drive. From inside this container, I can read and write to the host’s filesystem at the block device level. We’ll exploit this in the next lesson.”

```
# Another dangerous one: /dev/mem – physical memory access
$ docker run --rm --privileged alpine ls -la /dev/mem
crw-r----- 1 root kmem 1, 1 Jan 15 08:00 /dev/mem
```

“/dev/mem — direct access to physical memory. With the right capabilities and this device node, you can read and write the host’s RAM directly. This bypasses every software-level protection.”

[4:15-5:15] — Why Do Privileged Containers Exist? (PPT Slide)

SLIDE: Title: “Legitimate Use Cases for `--privileged`”

Use Case	Why Privileged Is Needed	Better Alternative
Docker-in-Docker (CI/CD)	Needs to create containers	Use kaniko, buildah, or mounted socket
System monitoring agents	Needs host-level visibility	Use specific capabilities + host PID/NET
Network plugins (CNI)	Needs to configure host networking	Use NET_ADMIN + NET_RAW only
Storage drivers	Needs block device access	Use specific device mounts
Hardware access (GPU)	Needs device files	Use <code>--device</code> flag for specific devices
Kernel module management	Needs SYS_MODULE	Manage modules on host directly

NARRATION:

“So why does `--privileged` exist? There are legitimate use cases — Docker-in-Docker for CI/CD pipelines, system monitoring agents that need host visibility, network plugins that configure host networking, and GPU workloads that need device access. But for every one of these, there’s a better alternative that doesn’t require full privilege. Docker-in-Docker? Use kaniko or buildah for rootless builds. Monitoring? Grant only the specific capabilities needed. GPU access? Use the `--device` flag to expose only the GPU device, not all devices. Network plugins? Use `--cap-add=NET_ADMIN` instead of `--privileged`. The `--privileged` flag is a sledgehammer when you almost always need a scalpel.”

[5:15-6:30] — Detecting Privileged Containers (Terminal + Code Editor)

NARRATION:

“As a security professional, you need to detect privileged containers in your environment.”

TERMINAL:

```
# Method 1: Docker inspect
$ docker run -d --name priv-test --privileged alpine sleep 3600
$ docker inspect priv-test --format '{{.HostConfig.Privileged}}'
true
```

“The simplest method — `docker inspect` with the Privileged field.”

```
# Method 2: Check from inside the container
$ docker exec priv-test sh -c 'cat /proc/1/status | grep -c
"CapBnd.*000001fffffffffff"'
```

“From inside a container, check the capability bounding set. If it’s all 1s, it’s privileged.”

```
# Method 3: List ALL running containers and check privilege status
$ docker ps -q | xargs -I{} docker inspect --format \
'{{.Name}}: Privileged={{.HostConfig.Privileged}}' {}
```

“To audit your entire environment — iterate through all running containers and check each one.”

```
# Clean up
$ docker rm -f priv-test
```

CODE EDITOR — Detection script at 150% zoom:

```
#!/bin/bash
# privileged_container_audit.sh
# Detect all privileged containers and containers with dangerous capabilities

echo "=== PRIVILEGED CONTAINER AUDIT ==="
echo "Date: $(date)"
echo "Host: $(hostname)"
echo ""

echo "--- Privileged Containers ---"
for cid in $(docker ps -q); do
    name=$(docker inspect --format '{{.Name}}' "$cid" | sed 's/\///')
    priv=$(docker inspect --format '{{.HostConfig.Privileged}}' "$cid")
    if [ "$priv" = "true" ]; then
        echo "[CRITICAL] $name ($cid) is running in PRIVILEGED mode"
    fi
done

echo ""
echo "--- Containers with Dangerous Capabilities ---"
DANGEROUS_CAPS="SYS_ADMIN SYS_PTRACE SYS_MODULE DAC_READ_SEARCH NET_ADMIN"
for cid in $(docker ps -q); do
    name=$(docker inspect --format '{{.Name}}' "$cid" | sed 's/\///')
    caps=$(docker inspect --format '{{.HostConfig.CapAdd}}' "$cid")
    for cap in $DANGEROUS_CAPS; do
        if echo "$caps" | grep -q "$cap"; then
            echo "[WARNING] $name has $cap capability"
        fi
    done
done

echo ""
echo "--- Containers with Docker Socket Mounted ---"
for cid in $(docker ps -q); do
    name=$(docker inspect --format '{{.Name}}' "$cid" | sed 's/\///')
    mounts=$(docker inspect --format '{{range .Mounts}}{{.Source}} {{end}}' "$cid")
    if echo "$mounts" | grep -q "docker.sock"; then
        echo "[CRITICAL] $name has Docker socket mounted"
    fi
done

echo ""
echo "=== AUDIT COMPLETE ==="
```

“Here’s a detection script you can use. It checks for privileged containers, dangerous capability additions, and Docker socket mounts — three of the most critical runtime misconfigurations. Run this as part of your regular security audit process.”

[6:30-7:15] — The Capability Deep Dive (PPT Slide)

SLIDE: Title: “The Most Dangerous Capabilities”

Capability	What It Allows	Escape Risk
CAP_SYS_ADMIN	Mount filesystems, configure namespaces, BPF, many more	CRITICAL — container escape via mount
CAP_SYS_PTRACE	Trace/control any process	HIGH — attach to host processes
CAP_SYS_MODULE	Load/unload kernel modules	CRITICAL — kernel-level code injection
CAP_DAC_READ_SEARCH	Bypass file read permission checks	HIGH — read any host file
CAP_NET_ADMIN	Configure networking	MEDIUM — network-level attacks
CAP_SYS_RAWIO	Raw I/O access	CRITICAL — direct hardware access
CAP_MKNOD	Create device files	HIGH — create device nodes for host devices

NARRATION:

“Not all capabilities are equally dangerous. SYS_ADMIN is the most versatile — it’s required for over 30 different privileged operations including mounting filesystems, which is the most common container escape technique. SYS_PTRACE allows process tracing — attaching a debugger to host processes if PID namespace is shared. SYS_MODULE allows loading kernel modules — game over if available. DAC_READ_SEARCH bypasses file permission checks. Any one of these in isolation can lead to a container escape under the right conditions. Privileged mode grants all of them simultaneously.”

[7:15-8:00] — Key Takeaways and Bridge (PPT Slide)

SLIDE: Title: “Key Takeaways”

1. `--privileged` disables ALL container security mechanisms simultaneously
2. It grants 41 capabilities, disables Seccomp, disables AppArmor, exposes all devices
3. A privileged container has equivalent access to a root process on the host
4. Every legitimate use case has a less-privileged alternative
5. Detection requires auditing both Docker inspect output and runtime behavior

NARRATION:

“The `--privileged` flag is the most dangerous single option in Docker. It disables every security mechanism simultaneously and gives the container root-equivalent access to the host. In the next lesson, we’re going to stop talking about what privileged containers can do — and actually do it. We’ll mount the host filesystem, read sensitive data, establish persistence, and demonstrate full host system compromise from inside a privileged container.”

SLIDE: “Next: Host System Compromise Demonstrations →”

SCRIPT END

Post-Production Notes

- Use red accent colors throughout this video for warning emphasis
- The capability comparison should use an animated side-by-side with default on left expanding to privileged on right
- For device listing, use split terminal: left showing default container, right showing privileged
- The detection script in the code editor should be syntax-highlighted with bash coloring
- Add a “ Lab Environment” watermark on all terminal demonstrations
- Capability table should have color-coded risk levels: red for CRITICAL, orange for HIGH, yellow for MEDIUM